

VALUTAZIONE DELL'EFFETTO DEL CONTESTO DI TEST SUL REFACTORING GENERATO DA LLM: UNO STUDIO SU APACHE AVRO

Matteo Lagioia

Ingegneria del Software II — A.A. 2024/2025
— *progetto individuale (Falessi).* Apache Avro
— *modulo lang/java/avro.* Classe target:
org.apache.avro.Schema. Baseline C0:
release-1.5.4. Repository: <https://github.com/StitchML/avro>. CI: <https://github.com/StitchML/avro/actions>.
SonarCloud: https://sonarcloud.io/project/overview?id=StitchML_avro. Data: 6 maggio 2026.

Abstract—Questo lavoro studia se fornire a un Large Language Model delle suite di test strutturate come parte del contesto di refactoring ne migliori in modo misurabile il comportamento. Su un hotspot di Apache Avro (`org.apache.avro.Schema`, release 1.5.4), abbiamo costruito un dataset class-level auditabile su 43 release (3 183 righe, 34 feature, 14,5 % di positivi, Linkage Rate Jira<->Git 40,1 %) usando SZZ leggero con fallback Proportion (P medio calibrato 0,9805, applicato al 23,3 % dei ticket con fix commit privi di `affectedVersions`). Abbiamo poi valutato in 10-times 10-fold cross-validation su quattro varianti del dataset (A, B, B+, C) tre classificatori Weka (Naive Bayes, Random

e una serie di vincoli interni (nomi univoci, validazione dei tipi, gestione di null). Questa varietà è ideale per costruire suite di test eterogenee: una Category Partition ragionata sulla classe aveva già prodotto, prima di M3, un corpus ampio di casi nominali, limite ed errore, pronto per essere tradotto in classi JUnit.

La versione baseline scelta è `release-1.5.4`. Due motivazioni giustificano questa decisione. La prima è la coerenza col perimetro storico del dataset M1/M2: il labeling di bugginess è costruito sull'intera timeline Avro osservabile (43 release, da `release-1.0.0` a `release-1.11.5`) mentre l'emissione delle righe è limitata al primo 33 % delle release in ordine cronologico, cioè la finestra `release-1.0.0 ... release-1.5.4`. Scegliere `release-1.5.4` come C0 significa scegliere l'estremo destro di questa finestra: la classe `Schema` viene riscritta esattamente nella release più recente in cui figura nel dataset. La seconda è operativa: la versione 1.5.4 contiene già il modulo Maven `lang/java/avro` nella forma che il progetto assume fino alla 1.8, con la classe `Schema.java` nel path standard `lang/java/avro/src/main/java/org/apache/avro/Schema.java`.

Una precisazione va fatta sul criterio di unicità del target. La specifica consente di scegliere più di una classe, ma abbiamo optato consapevolmente per una sola. La ragione è che una sperimentazione a classe singola permette di spingere al limite la profondità dell'analisi (sei livelli di testing più integrazione, quattro varianti di refactoring LLM, coverage e mutation per ciascuna) senza diluire il budget di tempo su più target. Il rischio di sovraspecializzazione del risultato sul caso `Schema` è reale ed è discusso apertamente al capitolo VII.

III. Costruzione del dataset (M1)

Il dataset class-level che alimenta la milestone di machine learning è stato prodotto dal pipeline `MantiMetrics`, un'applicazione Java 17 organizzata in undici package a responsabilità singola che coprono risoluzione della configurazione (`cli`, `config`), pianificazione delle release

che a sua volta invoca `ModelEvaluator` per la fase di validazione e `WhatIfAnalyzer` per lo studio controfattuale. Il writer finale `DatasetReportPublisher` produce i CSV `results.csv`, `milestone2_summary.csv`, `feature_correlations.csv`, `what_if_summary.csv` più due figure riassuntive per ciascun dataset (otto immagini PNG complessive nella cartella `output/charts/`). La riproducibilità è garantita dal profilo di lancio `run-analysis.cmd`, che fissa seed (42), strategia (`cross-validation`), numero di run e fold (10x10), classe positiva (`yes`), policy SMOTE/FS e attributo `NSmells` come feature azionabile imposta dalla traccia; l'esecuzione registrata come `RunId=20260506-102634` è quella da cui provengono tutti i numeri riportati in questo capitolo e in Appendice A.2.

IV-B. Strategia di validazione: 10-times 10-fold cross-validation

La strategia di validazione adottata è 10-times 10-fold cross-validation stratificata sull'attributo classe `Buggy`. Ciascuno dei quattro dataset (A, B, B+, C) è partizionato in 10 fold stratificati di pari popolazione di positivi; il fold di test è ruotato sui 10 indici, e l'intero protocollo viene ripetuto 10 volte cambiando il seed di partizionamento (seed base 42, stream deterministico). Le metriche di test sono mediate sui 100 fit complessivi per ogni coppia (dataset, classificatore). Questa scelta supera la criticità della prima versione del progetto, in cui la validazione walk-forward sull'attributo temporale `ReleaseId`

colonna in top-venti il cui azzeramento corrisponde a un'azione di ingegneria praticabile (l'eliminazione degli smell via refactoring). Sui dataset derivati `B.arff`, `BPlus.arff` e `C.arff` il pipeline attiva in parallelo un secondo what-if che usa la feature numerica `zerable` a correlazione assoluta più alta (rispettivamente `TotalAuthors` per `B` e `B+` e `TotalChurn` per `C`); questa branch alternativa non è imposta dalla rubrica ma è utile come termine di paragone per quantificare quanto il segnale azionato dal refactoring (`NSmells`) sia inferiore al segnale azionato da feature di processo cumulative.

IV-E. Confronto fra i tre classificatori sui quattro dataset

Tre famiglie di classificatori Weka sono valutate in 10x10 CV su tutti e quattro i dataset: Naive Bayes (`weka.classifiers.bayes.NaiveBayes`), Random Forest (`weka.classifiers.trees.RandomForest`, configurazione di default) e IBk con `k=1` (`weka.classifiers.lazy.IBk`). La Tabella I sintetizza per ciascuna coppia (dataset, classificatore) le sette metriche aggregate come riportate in `results.csv`: Accuracy, Precision, Recall, F1, Kappa, AUC e NPofB20. Le righe in grassetto evidenziano il classificatore che vince la singola metrica all'interno del dataset.

Tabella I. Metriche aggregate 10x10 CV
(RunId=20260506-102634, positive class yes, FeatureSelection=none, Balancing=none).

Il quadro è netto. Random Forest domina il confronto su tutti e quattro i dataset: vince Accuracy, Precision, F1, Kappa e AUC ovunque; cede Recall a IBk solo su `B.arff` (di poco) e NPofB20 a IBk solo su `C.arff` (la distanza è 0,0073). Naive Bayes è significativamente più debole in tutte le configurazioni: Kappa fra 0,336 (`BPlus.arff`) e 0,617 (`C.arff`), Accuracy intorno all'83 % sui dataset rumorosi e al 92 % sul dataset smell-free `C`. IBk si colloca stabilmente fra i due, con Kappa fra 0,608 (`BPlus.arff`) e 0,713 (`C.arff`).

Sul piano qualitativo i tre modelli esplorano trade-off complementari. Random Forest sfrutta a fondo la non-linearità delle interazioni fra le metriche di processo cumulative (`TotalAuthors`, `TotalTouches`, `TotalChurn`) e la miscela di metriche statiche e flag smell binari, ottenendo un equilibrio molto favorevole tra Precision e Recall. Naive Bayes paga l'assunzione di indipendenza condizionale fra feature, che è particolarmente fragile in un dataset ricco di metriche di processo correlate, e il suo Kappa più basso è in linea con questa diagnosi. IBk reagisce bene alla struttura locale dello spazio delle feature e ottiene Recall e NPofB20 paragonabili a RF, ma è penalizzato in Precision dalla volatilità del nearest-neighbor su dati rumorosi.

IV-F. Scelta del winner overall (Kappa -> AUC) e lettura rispetto alla soglia [0,1 ; 0,4]

Il criterio di selezione del classificatore winner imposto dal corso è Kappa come metrica principale e AUC come tie-breaker. Applicando questa regola alla 10x10 CV, l'OVERALL_WINNER registrato in `milestone2_summary.csv` per ognuno dei quattro dataset è **Random Forest** (Tabella II): Kappa fra 0,7073 (`BPlus.arff`) e 0,8286 (`C.arff`), AUC fra 0,9379 e 0,9553. Il tie-breaker AUC non si attiva mai, perché Random

Forest ha sempre il Kappa più alto a parità di dataset.

Tabella II. Overall winner per dataset (criterio Kappa-then-AUC).

L'interpretazione del valore di Kappa rispetto alla soglia di sanità prescritta dalla rubrica [0,1 ; 0,4] è duplice. La rubrica richiama questa soglia come controllo di non-casualità del dataset (M1) ed è utile rileggerla qui ora che la 10x10 CV stratificata ha rimosso le distorsioni da fold degeneri presenti nella precedente esecuzione walk-forward. Il classificatore che cade naturalmente nel range è **Naive Bayes**, con Kappa fra 0,3355 e 0,3505 sui tre dataset rumorosi (`A`, `B`, `BPlus`) e Kappa 0,6165 su `C`, fuori dal range solo perché `C` è il dataset più *ñpulitož* (smell-free) e quindi facile da classificare. **Random Forest** e **IBk** stanno sopra il limite superiore 0,4 in tutti i dataset, ma il loro essere fuori-range non è più — come nella vecchia versione walk-forward — un artefatto di aggregazione: è il riflesso autentico della maggiore capacità modellistica delle due famiglie su un dataset stratificato. La lettura aggregata è dunque positiva su entrambi i fronti: (i) il dataset M1 supera il controllo di sanità non-casualità (Naive Bayes, modello probabilistico minimale, fa Kappa positivo dentro il range); (ii) il classificatore winner (Random Forest) realizza un Kappa elevato senza dipendere da artefatti di aggregazione, ed è dunque selezionabile con confidenza per il what-if.

IV-G. Analisi what-if sulla feature azionabile NSmells

Sulla base del winner eletto, `WhatIfAnalyzer` con il supporto di `WhatIfPredictionEvaluator` e `WhatIfImpactCalculator` riaddestra Random Forest sui quattro dataset derivati da M1 (`A`, `B+`, `B`, `C`) e li confronta in forma aggregata. Sul dataset principale `A.arff` la feature azionabile imposta dalla traccia è `NSmells`. La Tabella III riassume gli scenari base (RowType=SCENARIO in `what_if_summary.csv`, righe relative a Dataset=`A.arff`, Feature=`NSmells`).

Tabella III. Scenari base per il what-if sul dataset principale A.arff, feature NSmells.

La lettura è la seguente. Nello scenario A (dataset pieno) Random Forest predice 434 buggy contro 462 reali, con probabilità positiva media 14,40 %: il modello è leggermente sottopredittivo sul totale ma molto vicino al ground truth (deviazione 6 %). Nello scenario B+ (le sole classi con `NSmells > 0`) RF predice 342 su 366 reali, con probabilità media 15,32 %: il modello è quasi calibrato sul sottoinsieme *ña* rischiož. Lo scenario B (controfattuale di B+ con le sette colonne azionabili portate a zero) si ferma a 340 predette buggy sugli stessi 366 reali, con probabilità media leggermente *augmentata* a 15,48 %; questo è un fenomeno noto del classificatore Random Forest, che non è strettamente monotono rispetto ai valori delle feature azionabili e può, su una piccola minoranza di istanze, alzare la probabilità positiva quando una feature azionata viene azzerata. Lo scenario C (le classi già smell-free in `A.arff`) predice 92 contro 96 reali, con probabilità media 11,79 %: questo livello di rischio predetto, vicino a quello dello scenario B, conferma che — per Random Forest — l'azzeramento delle feature azionabili sul sottoinsieme B+ produce una distribuzione di probabilità simile a quella del campione naturalmente smell-free.

Dataset	Classificatore	Accuracy	Precision	Recall	F1	Kappa	AUC	NPofB20
A.arff	Random Forest	94,40 %	0,9304	0,6651	0,7734	0,7426	0,9514	0,7583
A.arff	Naive Bayes	83,65 %	0,4418	0,4545	0,4459	0,3505	0,7667	0,2297
A.arff	IBk (k = 1)	92,38 %	0,7968	0,6409	0,7079	0,6648	0,8971	0,6873
B.arff	Random Forest	93,34 %	0,9090	0,6367	0,7457	0,7091	0,9383	0,7135
B.arff	Naive Bayes	83,44 %	0,4678	0,4246	0,4421	0,3458	0,7605	0,2418
B.arff	IBk (k = 1)	92,29 %	0,8239	0,6464	0,7212	0,6774	0,8903	0,6978
BPlus.arff	Random Forest	93,28 %	0,9043	0,6367	0,7443	0,7073	0,9379	0,7182
BPlus.arff	Naive Bayes	83,44 %	0,4660	0,4066	0,4311	0,3355	0,7560	0,2285
BPlus.arff	IBk (k = 1)	90,72 %	0,7652	0,5864	0,6608	0,6085	0,8765	0,6484
C.arff	Random Forest	96,88 %	0,9658	0,7629	0,8453	0,8286	0,9553	0,9079
C.arff	Naive Bayes	92,02 %	0,6716	0,6740	0,6609	0,6165	0,9050	0,8582
C.arff	IBk (k = 1)	94,16 %	0,7727	0,7369	0,7458	0,7131	0,9480	0,9152

Dataset	Winner	Kappa	AUC
A.arff	Random Forest	0,7426	0,9514
B.arff	Random Forest	0,7091	0,9383
BPlus.arff	Random Forest	0,7073	0,9379
C.arff	Random Forest	0,8286	0,9553

Scenario	Instances	Actual Buggy	Predicted Buggy	Avg P(buggy)
A	3 183	462	434	14,40 %
B+	2 355	366	342	15,32 %
B	2 355	366	340	15,48 %
C	828	96	92	11,79 %

IV-H. Impatto quantitativo del refactoring ideale ($B+ \rightarrow B$)

L'impatto sostantivo del refactoring, ovvero il confronto pairwise tra $B+$ e B sotto azionamento di NSmells, è riassunto dalla riga `IMPACT, B+ -> B` di `what_if_summary.csv` con `Dataset=A.arff`, `Feature=NSmells`, `Classifier=Random Forest` (Tabella IV).

Tabella IV. Impatto pairwise $B+ \rightarrow B$ (dataset A.arff, feature NSmells, classificatore winner: Random Forest).

La semantica delle colonne va letta con cura. `PredictedRelieved = 2` è il numero di istanze che il classificatore predice buggy in $B+$ e che, dopo l'azzeramento delle feature azionabili, vengono riclassificate clean in B : il modello *ícredez* che esattamente queste 2 istanze migliorino. `AvoidableBuggy = 2` è il sottoinsieme di quelle 2 istanze che sono anche effettivamente buggy secondo l'oracolo di M1 (label `Buggy=yes` nel dataset originale): quindi entrambe le redenzioni predette da Random Forest coincidono con bug reali, contro un solo bug effettivamente evitabile della precedente esecuzione walk-forward con winner Naive Bayes. `AvoidableBuggyShare = 2/366` circa 0,55 % è la normalizzazione di questa quantità sul totale di buggy effettivi in $B+$. La variazione media della probabilità positiva sulle coppie appaiate è pari a -0,16 %: leggermente negativa, conseguenza del fatto che su una minoranza di istanze RF non è strettamente monotono rispetto a NSmells.

La risposta sintetica alla domanda della traccia *íquanti metodi buggy si sarebbero potuti evitare se il codice fosse stato perfetto dal punto di vista della manutenibilità?* è dunque, in questa esecuzione, **due istanze di classe buggy effettivamente evitabili** azzerando l'unica feature azionabile imposta dalla traccia (NSmells). Il numero è coerente con la modesta correlazione di NSmells con Buggy nel dataset Avro (0,10,

un quarto di quella di TotalAuthors) e con il fatto che il rischio di bug in Avro è guidato in larga parte dalle metriche di processo cumulative, non azionabili tramite refactoring sintattico. Va letto come un *lower bound* del beneficio reale del refactoring per due ragioni: (i) NSmells è una sola colonna su ventotto e non rappresenta interamente lo spazio degli smell strutturali, (ii) Random Forest è non-monotono e può conservare predizioni yes anche dopo l'azzeramento, sotto-stimando le redenzioni effettive.

IV-I. What-if alternativi e confronto fra feature azionate

Il pipeline attiva un secondo blocco di what-if sui dataset B.arff, BPlus.arff e C.arff usando la feature numerica zerabile a correlazione assoluta più alta come driver (*íhighest absolute correlation among zeroable numeric features* nella colonna `FeatureSelectionReason`). La Tabella V riassume i risultati di questa branch alternativa, utile per quantificare l'ordine di grandezza del beneficio del refactoring quando l'azione viene fatta su una feature di segnale forte invece che su una feature debole come NSmells.

Tabella V. What-if alternativi (riga `IMPACT, B+ -> B` per dataset, classificatore Random Forest).

Il quadro è eloquente. Azionare NSmells (l'unica feature realmente azionabile via refactoring di codice) elimina 2 bug. Azionare TotalAuthors, che ha correlazione $|r|=0,411$ con Buggy (rank 1), eleverebbe la stima a 29-35 bug evitabili sul sottoinsieme $B+$ di 366 reali: l'ordine di grandezza è 15x quello del refactoring. Tuttavia TotalAuthors non è azionabile via refactoring; ridurla a zero significherebbe rimuovere autori dalla storia del file, che è una operazione *ícontabile*, non una intervento di ingegneria del software. La tabella V chiarisce dunque, in modo numerico, il punto

Metrica	Valore
Instances paired	2 355
Actual Buggy	366
Predicted Relieved	2
Avoidable Buggy	2
Avoidable Buggy Share	0,55 % (2/366)
delta P(buggy) media	-0,16 %

Dataset	Feature azionata	Paired	Predicted Relieved	Avoidable Buggy	Share	delta P(buggy) media
A.arff	NSmells	2 355	2	2	0,55 %	-0,16 %
B.arff	TotalAuthors	2 096	29	29	7,92 %	1,51 %
BPlus.arff	TotalAuthors	2 096	35	35	9,56 %	1,78 %
C.arff	TotalChurn	426	4	4	4,49 %	-0,001 %

centrale dell’analisi: il beneficio del refactoring, misurato sull’unico canale azionabile dalla traccia (NSmells), è significativamente più piccolo del segnale teoricamente disponibile guardando le feature più correlate a Buggy. Il refactoring resta uno strumento utile, ma il ritorno medio nella popolazione globale di Avro è modesto e l’effort va concentrato sui veri hotspot — che è esattamente la motivazione di M3 e della scelta di `org.apache.avro.Schema` come singola classe target.

IV-J. Figure riassuntive

Il pipeline produce, per ciascuno dei quattro dataset, due figure: un *bar chart* delle sette metriche aggregate per i tre classificatori (`avro_dataset_<NAME>_bar.png`) e un *boxplot* della loro distribuzione fold-per-fold (`avro_dataset_<NAME>_box.png`), per un totale di otto immagini riferite in Appendice A.2.4. I bar chart confermano visivamente la dominanza di Random Forest su tutte le metriche e il distacco di Naive Bayes nelle tre dimensioni più sensibili (Precision, Recall, Kappa). I boxplot mostrano una varianza inter-fold contenuta per RF e IBk, indicativa di stabilità del modello sui 100 fit; la varianza più alta è per Naive Bayes su Recall e F1, coerente con la fragilità dell’ipotesi di indipendenza in presenza di feature di processo correlate.

V. Metodologia M3 e protocollo di refactoring LLM

La milestone 3 trasforma la selezione in ingegneria di test. La baseline C0 è stata congelata con un fork di `apache/avro` sotto il profilo `StitchML/avro`, il checkout del tag `release-1.5.4` e la creazione del branch `m3-c0-schema`. Il modulo target `lang/java/avro` è stato verificato compilabile con Maven e Temurin 8, e la suite di test originali è stata disabilitata non per eliminazione ma per esclusione esplicita a livello di configurazione Surefire, mediante pattern di include limitati ai file `*M3*.java` per la componente unit e `*M3*IT.java` per la componente di integrazione. Questa scelta preserva i test originali come ispezione futura e rende trasparente al lettore della pipeline quale sottoinsieme di test governa l’esito del CI.

La Continuous Integration è implementata come workflow GitHub Actions `.github/workflows/m3-ci.yml`, attivato a ogni push e pull request sul fork, e consiste in un job Linux che effettua il checkout, il setup di Temurin 8, la cache delle dipendenze Maven, la compilazione del modulo, l’esecuzione di unit e IT, e il caricamento dei report Surefire

e Failsafe come artefatti. Le estensioni previste per JaCoCo e PIT sono compatibili con la stessa pipeline e vengono attivate quando necessario nelle esecuzioni manuali.

La suite di test su C0 è multilivello e segue le sei tecniche richieste dalla traccia M3. La componente `black-box` è `SchemaM3BBTest` con 18 casi, derivata direttamente dalla `Category Partition` esistente sulla classe e organizzata per gruppi (factory methods; validazione e casi d’errore; comportamento specifico record; union ed enum; parsing deprecato; equality e rappresentazione testuale). La componente `random` è `SchemaM3RandomTest` con 12 casi, ottenuti tramite Randoop con target limitato alla sola `org.apache.avro.Schema`, timeout corto per contenere l’esplosione di test e successiva sanitizzazione manuale per scartare casi flaky o non deterministici. La componente LLM è `SchemaM3LLMTest` con 8 casi, prodotti tramite prompt engineering strutturato che specifica package, vincoli di compilabilità, determinismo, assenza di mocking non necessario, e focus methods. La componente `coverage-guided` è `SchemaM3CoverageTest` con 11 casi, scritti iterativamente sui buchi evidenziati da JaCoCo dopo l’esecuzione delle prime tre componenti. La componente `control-flow` è `SchemaM3CFTTest` con 17 casi, costruiti su CFG semplificati di cinque metodi critici (`createUnion`, `createRecord`, `createEnum`, `setFields`, `equals`) annotando branch significativi e attraversandoli uno a uno. La componente `mutation-guided` è `SchemaM3MutationTest` con 16 casi, progettati come killer dedicati per survivor PIT selezionati per interesse, con focus su boundary, null-check, boolean returns e branch di validazione.

A queste sei componenti si aggiunge `SchemaM3IT` con 8 casi di integrazione eseguiti dal plugin Failsafe, che utilizzano Mockito 1.10.19 in configurazione sia `spy()` sia `mock()` per esercitare le interazioni di `Schema` con componenti simulati senza rompere la visibilità del package. Il totale su C0 è 82 test unit più 8 test IT, per 90 test complessivi, tutti passing, zero failure, zero error, zero skip.

Il protocollo di refactoring LLM è costruito per isolare l’effetto del contesto fornito al modello. Quattro varianti C1, C2, C3, C4 partono tutte dal branch C0 (non in cascata una dall’altra) così che ciascuna variante sia confrontabile con C0 in modo diretto. Per ogni variante abbiamo interrogato in parallelo due LLM come alternative provider: GPT-5.4 e Claude Opus 4.6; per ciascuna variante abbiamo selezionato la ver-

sione che superava l'intera suite C0 con il minimo LOC delta e, in caso di parità, la variante con complessità ciclomatica più bassa. La variante C1 riceve soltanto il sorgente originale con diagnostica smell. La variante C2 aggiunge al contesto di C1 l'intera suite black-box SchemaM3BBTest, così che il modello abbia una specifica comportamentale visibile. La variante C3 aggiunge alla C2 la suite control-flow SchemaM3CFTTest, introducendo pressione sui percorsi interni. La variante C4 aggiunge infine la lista dei mutanti sopravvissuti dall'esecuzione PIT su C0, con descrizione testuale per ciascun survivor. I vincoli comuni imposti ai modelli in ogni variante sono il divieto di modificare l'API pubblica, il divieto di modificare i test, la compatibilità con Java 8, la preservazione dei nomi di package e dei file originali, e il rifiuto di introdurre dipendenze non già presenti nel pom.xml di lang/java/avro.

VI. Risultati e discussione

by-line sui bug-inducing commit; eventuali linee rinominate o spostamenti cross-file sono ereditati dal nostro dataset e non vengono propagati indietro oltre la granularità di file. Il Linkage Rate osservato (442 su 1 101, circa 40,1 %) è allineato ai valori tipici della letteratura per progetti Apache ma lascia aperta una quota del 60 % di ticket non agganciati: questa quota non introduce falsi positivi (il recall dell'oracolo scende, la precision no) ma sposta verso il basso la proporzione reale di righe buggy. Un terzo elemento è la granularità del dataset: la specifica del corso formula l'analisi what-if in termini di metodi buggy ma l'estrazione M1 è stata eseguita a livello di classe, per ragioni di costo computazionale e di stabilità del fit del P. I numeri riportati nel capitolo IV vanno quindi letti come istanze di classe, non come metodi: l'ordine di grandezza dei delta what-if è verosimilmente simile, ma la riproduzione method-level richiederebbe un secondo pipeline di estrazione e un ricalcolo della calibrazione Proportion. Un quarto elemento è il detector di code smell usato per popolare NSmells, SmellDensity e i quattro flag binari: la traccia richiede esplicitamente SonarCloud, mentre qui la stessa definizione operativa è implementata in-house a partire da detector JavaParser più un rilevatore di duplicazione a token (CloneDetector del package clone). Questa deviazione è stata fatta per garantire la riproducibilità offline della pipeline, che deve poter essere eseguita senza autenticazione OAuth e senza esaurire la quota free di SonarCloud su 43 scansioni.

Sulla validità interna, la limitazione maggiore è la singola classe target. Un esperimento a classe singola non permette generalizzazione al modulo o al progetto. La scelta legacy di release-1.5.4 e la toolchain Java 8 con Mockito 1.10.19 introducono una possibile non-rappresentatività rispetto a code-base moderni. Il protocollo di prompt engineering usato per C2, C3 e C4 non è stato sottoposto ad ablation sistematica. La sanitizzazione manuale della componente random è basata su euristiche che l'autore ha applicato in modo coerente ma non è perfettamente replicabile da un terzo.

Sulla validità esterna, il progetto è uno solo e l'esperimento riguarda un unico modulo Java di apache/avro. I due LLM usati (GPT-5.4 e Claude Opus 4.6) sono rappresentativi dello stato dell'arte di aprile 2026 ma la frontiera si muove rapidamente; un replicato con modelli diversi o successivi potrebbe invertire la gerarchia tra varianti.

Sulla validità di conclusione, novanta test non sono un campione sufficiente a supportare test statistici di significatività su differenze piccole fra varianti. Il mutation score calcolato con PIT su un bytecode Java 8 legacy può includere mutanti equivalenti non eliminabili, che gonfiano artificiosamente il numero di survivor. La coverage di linee al 45–48 % su una classe della scala di Schema è un numero intermedio: sufficiente per leggere le differenze tra varianti, ma troppo basso per escludere che un investimento massiccio in test aggiuntivi cambierebbe il quadro. Per quanto riguarda M2, la 10x10 CV stratificata adottata in questa edizione del run rimuove la principale criticità dell'esecuzione walk-forward precedente (8 fold su 13 con test set privi di positivi, Kappa medio inflazionato), ma due riserve restano. Primo, la 10x10 CV non rispetta l'ordine temporale delle release: in produzione il classificatore vedrebbe solo dati passati per predire release future, e questa proprietà non è preservata dal partizionamento stratificato; il pipeline supporta uno split 80/20 ordinato attivabile via --split=ordered-80-20 come controllo

di realismo temporale, ma in questa esecuzione non è stato eseguito perché il RunId=20260506-102634 riporta solo ValidationStrategy=cross-validation. Secondo, la baseline è FeatureSelection=none e Balancing=none: le combinazioni con FS via CfsSubsetEval o con SMOTE applicato sul solo training set (entrambe supportate dal FilteredClassifier) potrebbero modificare la gerarchia tra classificatori e la stima di AvoidableBuggy; un'ablation FSxBalancing è il primo intervento nell'agenda di lavoro futuro.

VIII. Conclusioni

La domanda di ricerca era se fornire a un LLM delle suite di test strutturate come parte del contesto migliori il refactoring behavior-preserving che il modello produce. La risposta breve è sì, con qualificazioni. Il contesto nsorgente più specifica black-box (variante C2) è il punto di maggior rendimento: produce refactoring che passano tutti gli oracoli, riducono LOC, e mantengono coverage e mutation score al livello della baseline. L'aggiunta del contesto control-flow (C3) conferma ma non amplifica questo beneficio, segnalando che la specifica comportamentale contiene già la maggior parte dell'informazione utile. L'aggiunta dei mutanti sopravvissuti (C4) non aiuta e può peggiorare, in quanto spinge il modello a riscritture motivate da un segnale fuori-dominio rispetto al nostro oracolo.

Sul piano dei numeri di M2, l'analisi comparativa dei tre classificatori Weka (Naive Bayes, Random Forest, IBk) in 10-times 10-fold cross-validation stratificata sui quattro dataset (A, B, B+, C) con FeatureSelection=none e Balancing=none elegge **Random Forest** come winner overall su tutti e quattro i dataset (Kappa fra 0,707 e 0,829, AUC fra 0,938 e 0,955). Naive Bayes cade dentro il range nominale di sanità Kappa [0,1 ; 0,4] sui tre dataset rumorosi (A, B, B+) come atteso da un modello probabilistico minimale, mentre RF e IBk superano il limite superiore in modo legittimo grazie alla maggiore capacità modellistica e al partizionamento stratificato. Lo studio what-if sull'unica feature azionabile imposta dalla traccia (NSmells), eseguito sul dataset principale A.arff con winner Random Forest, stima in **due istanze di classe** il nbug effettivamente evitabile grazie a un refactoring ideale che azzerava la feature, su un totale di 366 buggy reali nel sottoinsieme B+ (share 0,55 %, delta P media -0,16 %). I what-if alternativi sui dataset B, B+, C, attivati sulla feature numerica zerabile a correlazione assoluta più alta (TotalAuthors per B e B+, TotalChurn per C), portano la stima a 29-35 bug evitabili, ma azionano feature di processo storico non riducibili a un'azione di refactoring di codice. La conclusione operativa è che il beneficio del refactoring sintattico, misurato sull'unico canale azionabile dalla traccia, è significativamente più piccolo del segnale teoricamente disponibile in feature non azionabili: il refactoring resta utile, ma il ritorno medio nella popolazione globale di Avro è modesto e l'effort va concentrato sui veri hotspot.

Per un contesto di ricerca futuro, le direzioni sono quattro. Primo, estendere l'esperimento a un piccolo insieme di classi hotspot Avro (due o tre) per verificare che il pattern osservato su Schema sia generalizzabile. Secondo, replicare con modelli LLM di generazioni diverse, in particolare con un'ablation su C2 che isoli quale sotto-porzione della suite black-box effettivamente produce il beneficio. Terzo, eseguire un'ablation FSxBalancing in M2 (CfsSubsetEval / Wrapper-BestFirst x Undersam-

pling / Oversampling / SMOTE) e affiancarla con uno split 80/20 ordinato come controllo di realismo temporale rispetto alla 10x10 CV stratificata. Quarto, sostituire il fallback Proportion a P medio con una calibrazione Cold o Incremental (P calcolato su finestre temporali chiuse) e ripetere la scelta dell'hotspot, per verificare che Schema resti il candidato di riferimento una volta rimossa la scorciatoia metodologica.

Più in generale, questo lavoro suggerisce che l'integrazione tra tecniche classiche di software testing e generazione assistita da LLM è più fruttuosa quando i test sono trattati come specifica comportamentale piuttosto che come oracolo post-hoc. Un modello che ha visto, prima di generare la sua riscrittura, quali comportamenti il codice è tenuto a preservare, produce un refactoring più coeso con il contratto della classe. Il dato che il rendimento decresce aggiungendo informazione a grana più fine (mutation survivors) è consistente con una lettura cognitiva del problema: la specifica behavior-level è la granularità giusta perché coincide con la granularità a cui l'LLM ragiona quando legge il codice.

Appendici

A.1 — Derivazione dei dataset M1

Le sette colonne azionabili azzerate in B sono CodeSmells, NSmells, SmellDensity, isLongMethod, isGodClass, isFeatureEnvy, isDuplicatedCode, come elencato in DatasetColumns::actionableColumns(). La label Buggy è preservata in tutti e quattro i dataset.

A.2 — Metriche M2

A.2.1 Metric winners (10x10 CV, positive class yes, FeatureSelection=none, Balancing=none) milestone2_summary.csv riporta i metric-winner per ciascun dataset; sotto si elencano per il dataset principale A.arff. Sui dataset B, B+ e C Random Forest mantiene la dominanza su tutte le metriche tranne — su B.arff — Recall (vinta da IBk con 0,6464) e — su C.arff — NPofB20 (vinta da IBk con 0,9152).

Overall winner secondo il criterio Kappa -> AUC, per ciascun dataset: **Random Forest** ovunque (cfr. Tabella II in IV-F). Fonte: output/milestone2_summary.csv, RunId=20260506-102634.

A.2.2 What-if su NSmells (classificatore winner: Random Forest, dataset A.arff) Scenari base (riga RowType=SCENARIO in what_if_summary.csv, Dataset=A.arff, Feature=NSmells):

Impatto pairwise B+ -> B (riga RowType=IMPACT, Dataset=A.arff, Feature=NSmells):

What-if alternativi (riga RowType=IMPACT per Dataset {B.arff, BPlus.arff, C.arff}, feature scelta come massima correlazione assoluta tra le numeriche zerable, classificatore Random Forest):

A.2.3 Ranking di correlazione feature -> Buggy (top 15)

Estratto da output/feature_correlations.csv, Pearson su feature numeriche contro la label Buggy, valore assoluto ordinato decrescente:

NSmells (rank 20, 0,101) è l'unica feature con SelectedForWhatIf=true con reason preferred

exam feature 'NSmells' sul dataset principale A.arff. Sui dataset derivati il pipeline attiva un secondo what-if con la feature numerica zerable a correlazione assoluta più alta (TotalAuthors per B e B+, TotalChurn per C), con reason highest absolute correlation among

Dataset	Righe	Criterio di filtro	Valori azionabili
A	3 183	Intero dataset classifier-ready	Preservati
B+	2 355	Filtro: NSmells > 0 (classi con almeno uno smell)	Preservati
B	2 355	Stessa selezione di B+	Azzerati su 7 colonne azionabili
C	828	Filtro: NSmells == 0 (classi smell-free)	Preservati (tutti zero per costruzione)

Campo (milestone1-audit.json)	Valore
rows	3 183
featureCount	34
distinctReleasesInDataset	14
buggyRows / cleanRows / smellyRows	462 / 2 721 / 2 355
snoring.timelineReleaseCount	43
snoring.selectedReleaseCount	14
snoring.selectedPercentageOfTimeline	32,56 %
labeling.strategy	proportion-fallback
labeling.totalResolvedTickets	1 101
labeling.ticketsWithFixCommit	442
labeling.ticketsUsingAffectedVersions	287
labeling.ticketsUsingTotalFallback	103
labeling.notes (P calibrato)	mean P = 0,9805; fallback P = 1,0

Metrica	Winner (A.arff)	Valore
Accuracy	Random Forest	94,40 %
Precision	Random Forest	0,9304
Recall	Random Forest	0,6651
F1	Random Forest	0,7734
Kappa	Random Forest	0,7426
AUC	Random Forest	0,9514
NPofB20	Random Forest	0,7583

Scenario	Instances	Actual Buggy	Predicted Buggy	Avg P(buggy)
A	3 183	462	434	14,40 %
B+	2 355	366	342	15,32 %
B	2 355	366	340	15,48 %
C	828	96	92	11,79 %

Paired	Actual Buggy	Predicted Relieved	Avoidable Buggy	Avoidable Buggy Share	delta P(buggy) media
2 355	366	2	2	0,55 % (2/366)	-0,16 %

Dataset	Feature azionata	Paired	Predicted Relieved	Avoidable Buggy	Share	delta P(buggy) media
B.arff	TotalAuthors	2 096	29	29	7,92 %	1,51 %
BPlus.arff	TotalAuthors	2 096	35	35	9,56 %	1,78 %
C.arff	TotalChurn	426	4	4	4,49 %	-0,001 %

Rank	Feature	Corr	Tipo
1	TotalAuthors	0,411	processo cumul.
2	TotalTouches	0,365	processo cumul.
3	TotalIssueTouches	0,357	processo cumul.
4	TotalChurn	0,264	processo cumul.
5	IssueTouches	0,215	processo istant.
6	Touches	0,213	processo istant.
7	Authors	0,189	processo istant.
8	DeletedLines	0,182	processo istant.
9	Churn	0,170	processo istant.
10	AddedLines	0,158	processo istant.
11	Cyclomatic	0,141	statica
12	Cognitive	0,139	statica
13	DistinctOperands	0,137	Halstead
14	Vocabulary	0,136	Halstead
15	AgeInReleases	0,134	storica

Parametro	Valore
Strategia	10-times 10-fold cross-validation
Stratificazione	sull'attributo classe Buggy (positiva: yes)
Numero di run	10
Numero di fold per run	10
Seed di base	42 (incrementi deterministici per run)
Attributo NPofB20 size	LOC
Feature Selection	none (baseline; CfsSubsetEval o Wrapper - BestFirst attivabili via flag)
Balancing	none (baseline; SMOTE, SpreadSubsample, Resample attivabili via flag)
Sequenza di pre-elabor.	FS prima di Balancing (richiesto dalla rubrica)
Implementazione	weka.classifiers.meta.FilteredClassifier su training set di ciascun fold

Variante	Line cov. Schema	Mutation score	Test-strength	Note
C0	47 %	47 %	78 %	baseline, 90/90 test passing
C1	48 %	48 %	79 %	sorgente + diagnostica smell
C2	48 %	48 %	79 %	+ suite black-box SchemaM3BBTest
C3	48 %	48 %	79 %	+ suite control-flow SchemaM3CFTest
C4	45 %	45 %	78 %	+ lista survivor PIT

Variante	delta LOC vs C0	isLongMethod	isGodClass	isFeatureEnvy
C0	—	sì	sì	sì
C1	riduzione moderata	sì	sì	marginale OK
C2	riduzione favorevole	sì	marginale OK	sì
C3	riduzione favorevole	sì	marginale OK	sì
C4	riduzione moderata	sì	sì	sì